

	CSE 482L / ETE334 Internet and Web Technology Lab	Lab Manual
	Faculty: rifat.hassan02@northsouth.edu Instructor: kazi.tanvirakter@northsouth.edu Updated: 2026-June-16	1

Lab Outcomes

After completing this lab, students will be able to:

1. Understand version control concepts and use Git/GitHub effectively.
2. Integrate AI tools responsibly into the development workflow.
3. Utilize advanced VS Code features and extensions for web development.
4. Create basic structured web pages using HTML5 semantic tags and CSS.

1. Introduction to Git

1.1 What is Git?

- Git is a **distributed version control system (DVCS)** that tracks changes in source code during software development.
- Developed by **Linus Torvalds** in 2005.
- Allows multiple developers to work on the same project without overwriting each other's work.

Key Features:

- **Distributed:** every developer has a full copy of the repository.
- **Tracks history:** every change is stored with metadata (author, date, message).
- **Supports branching and merging** efficiently.

1.2 Why Use Git?

1. **Version History:** Keep track of every change in the code.
2. **Collaboration:** Multiple developers can work simultaneously.
3. **Backup:** Local and remote repositories prevent data loss.
4. **Branching & Experimentation:** Create branches for features or experiments without affecting main code.
5. **Integration with Platforms:** GitHub, GitLab, Bitbucket for hosting repositories and CI/CD integration.

2. Common Git Commands

Command	Purpose / Meaning	Example
git init	Initialize a new Git repository in the current folder	git init

Command	Purpose / Meaning	Example
<code>git status</code>	Check the status of files (tracked, untracked, staged)	<code>git status</code>
<code>git add <file></code>	Stage file(s) for commit	<code>git add index.html</code>
<code>git commit -m "message"</code>	Save staged changes to local repository with a message	<code>git commit -m "Initial commit"</code>
<code>git log</code>	View commit history	<code>git log</code>
<code>git diff</code>	Show changes between working directory and staged files	<code>git diff</code>
<code>git branch</code>	List, create, or delete branches	<code>git branch feature1</code>
<code>git checkout <branch></code>	Switch to another branch	<code>git checkout feature1</code>
<code>git merge <branch></code>	Merge a branch into the current branch	<code>git merge feature1</code>
<code>git clone <repo-url></code>	Copy a remote repository to local machine	<code>git clone https://github.com/user/repo.git</code>
<code>git pull</code>	Fetch and merge changes from remote repository	<code>git pull origin main</code>
<code>git push</code>	Send local commits to remote repository	<code>git push origin main</code>
<code>git remote -v</code>	List remote repositories	<code>git remote -v</code>
<code>git reset <file></code>	Unstage a file	<code>git reset index.html</code>
<code>git rm <file></code>	Remove a file from repo and stage the deletion	<code>git rm oldfile.txt</code>

3. Git Workflow

1. Initialize Repository:

```
git init
```

2. Check Status:

```
git status
```

3. Stage Changes:

```
git add <file>
```

4. Commit Changes:

```
git commit -m "Initial commit"
```

5. Create Branch (Optional):

```
git branch feature1  
git checkout feature1
```

6. Merge Branch to Main:

```
git checkout main  
git merge feature1
```

7. Connect Remote Repository:

```
git remote add origin <repo-url>  
git push -u origin main
```

8. Pull Changes from Remote:

```
git pull origin main
```

4. First Git Tasks for Students

Task 1: Initialize a Local Repository

- Create a folder called MyProject.
- Open terminal and run:

```
git init
```

- Create index.html with a heading "Hello Git!".
- Stage and commit the file:

```
git add index.html  
git commit -m "Add initial HTML file"
```

Task 2: Create a Branch and Make Changes

- Create a new branch:

```
git branch feature-update  
git checkout feature-update
```

- Update index.html by adding a paragraph.
- Stage and commit changes:

```
git add index.html
```

```
git commit -m "Add introduction paragraph"
```

- Merge branch back to main:

```
git checkout main  
git merge feature-update
```

Task 3: Connect and Push to GitHub

- Create a new repository on GitHub (e.g., MyProject).
- Connect remote:

```
git remote add origin https://github.com/username/MyProject.git  
git push -u origin main
```

- Verify files on GitHub and update commit history.

5. Good Practices

- Commit frequently with meaningful messages.
- Use `.gitignore` for files that should not be tracked (`node_modules`, `.env`).
- Always pull changes before starting work:

```
git pull origin main
```

- Use branches for new features or experiments.
- Keep commit messages clear and descriptive.

6. Understanding the Structure of an HTML Document

HTML defines the structure of web content using elements enclosed in tags.

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>My First Web Page</title>  
</head>  
<body>  
  <h1>Welcome!</h1>  
  <p>This is my first web page.</p>  
</body>  
</html>
```

Key Points:

- `<!DOCTYPE html>` declares HTML5 version.
- `<html>`: root of the page.
- `<head>`: metadata (title, charset, stylesheet links).
- `<body>`: visible page content.

Good Practice Tasks:

1. Create a basic page that shows your name, student ID, and department.
2. Add a horizontal line `<hr>` and use `<br>` for spacing.
3. Try changing `<title>` and refresh your browser tab.

7. HTML Attributes

HTML attributes provide additional information about elements. They are always written in the opening tag, using the format: `attribute="value"`

```

```

Common Global Attributes

Attribute	Description	Example
id	Unique identifier for an element	<code><p id="intro">Text</p></code>
class	Group similar elements for styling	<code><p class="note">Note</p></code>
title	Tooltip text shown on hover	<code><abbr title="HyperText Markup Language">HTML</abbr></code>
style	Inline CSS styling	<code><h1 style="color:blue;">Heading</h1></code>
lang	Language of the page	<code><html lang="en"></code>
dir	Text direction	<code><p dir="rtl">Right-to-left text</p></code>
hidden	Hides element	<code><p hidden>This is hidden</p></code>
tabindex	Sets tab navigation order	<code><input tabindex="1"></code>

Tag-Specific Attributes

Tag	Important Attributes	Example
<code><a></code>	href, target, title	<code>Visit</code>
<code></code>	src, alt, width, height	<code></code>
<code><table></code>	border, cellpadding, cellspacing	<code><table border="1"></code>
<code><form></code>	action, method	<code><form action="submit.php" method="post"></code>
<code><input></code>	type, name, value, placeholder	<code><input type="text" placeholder="Enter name"></code>

```
<a href="https://www.google.com" target="_blank" title="Go to Google">Google</a>

<p style="color:green;" id="info" class="highlight">Welcome to HTML Lab!</p>
```

Practice Tasks:

1. Create a small HTML page that includes at least 10 tags using different attributes.
Example: image, link, paragraph, list, and form inputs.
2. Add a tooltip (title) on your links and images.
3. Assign id and class to at least three elements for styling later with CSS.
4. Experiment with lang, hidden, and dir attributes to see their behavior.

Best Practices

- Attribute names are case-insensitive, but use lowercase for consistency.
- Always enclose values in quotes.
- Always include alt for image description and href for links.
- Avoid inline styles unless necessary.

8. Text Formatting and Headings

HTML offers many tags for text emphasis and readability.

Tag	Function	Example
<h1>—<h6>	Headings	Subsection
<p>	Paragraph	This is text.
 / 	Bold text	Important!
<i> / 	Italic / emphasized text	Note this
<u>	Underline	<u>HTML is cool!</u>
<mark>	Highlight text	<mark>Highlighted</mark>
<small>	Smaller font	Fine print
<sub> / <sup>	Subscript/Superscript	H₂O
<blockquote>	Quotation block	<blockquote>Quote text</blockquote>
<pre>	Preformatted text	<pre>code block</pre>

Practice Tasks:

1. Create a text-based "About Me" page using at least 10 formatting tags.
2. Add a famous quote using <blockquote>.
3. Use <pre> to show code snippets or poems.

9. Lists, Links, and Navigation

Lists organize content; links connect pages and sections. There 3 types of lists:

1. Unordered List
2. Ordered List
3. Definition List

```

<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>
<ol>
  <li>Wake up</li>
  <li>Study</li>
</ol>
<dl>
  <dt>HTML</dt>
  <dd>A markup language</dd>
</dl>

```

Links

```

<a href="https://www.google.com" target="_blank">Visit Google</a>
<a href="#top">Go to top of page</a>

```

Attributes:

- href – URL or anchor
- target="_blank" – opens in new tab
- title – tooltip

Good Practice Tasks:

1. Create a "Table of Contents" that links to sections within your own page using internal anchors (id).
2. Make a navigation bar using a list of hyperlinks.

10. Images and Multimedia

Images and media make content visual. Use relative paths (e.g., images/photo.jpg) instead of absolute ones.

```


<figure>
  
  <figcaption>Beautiful Flower</figcaption>
</figure>
<audio controls>
  <source src="music.mp3" type="audio/mpeg">
</audio>
<video width="400" controls>
  <source src="clip.mp4" type="video/mp4">
</video>

```

Good Practice Tasks:

1. Create a photo gallery using <figure> and <figcaption>.
2. Add background music using <audio>.
3. Write a paragraph describing image copyright and use.

11. Tables for Data Representation

Tables present structured information in rows and columns.

Tag	Function
<table>	Defines a table
<tr>	Table row
<th>	Header cell
<td>	Data cell
<caption>	Table title
colspan, rowspan	Merging cells

```
<table border="1">
  <caption>Student Marks</caption>
  <tr><th>Name</th><th>HTML</th><th>CSS</th></tr>
  <tr><td>Ali</td><td>85</td><td>90</td></tr>
</table>
```

Practice Tasks:

1. Create a timetable using merged cells.
2. Center table content using the align attribute.
3. Add a caption to your table.

12. Forms and User Input

Forms collect user data for submission or interaction.

Common Tags

Tag	Purpose
<form>	Defines a form
<input>	Input fields
<label>	Label for inputs
<textarea>	Multi-line input
<select> + <option>	Dropdown menu
<fieldset> + <legend>	Group fields
<button>	Button element

```
<form>
  <fieldset>
    <legend>Registration Form</legend>
    <label>Name:</label><input type="text" placeholder="Your Name"><br>
    <label>Email:</label><input type="email"><br>
    <label>Password:</label><input type="password"><br>
```



```

<label>Gender:</label>
<input type="radio" name="g" value="m">Male
<input type="radio" name="g" value="f">Female<br>
<input type="checkbox" checked> Subscribe to newsletter<br>
<input type="submit" value="Submit">
</fieldset>
</form>

```

Practice Tasks:

1. Create a complete feedback form with text, email, and dropdown inputs.
2. Use <fieldset> and <legend> for grouping.
3. Validate fields with required attributes.

13. Semantic HTML and Page Layout

Semantic tags improve page organization and readability.

Tag	Meaning
<header>	Page header or title
<nav>	Navigation menu
<section>	Thematic grouping
<article>	Independent content
<aside>	Sidebar
<footer>	Footer content

```

<header><h1>Student Blog</h1></header>
<nav>
  <a href="#">Home</a> | <a href="#">Posts</a>
</nav>
<section>
  <article>
    <h2>Lab Experiences</h2>
    <p>This week I learned about forms...</p>
  </article>
</section>
<footer>&copy; 2025 Student Blog</footer>

```

Practice Tasks:

1. Build a news layout with multiple <article> sections.
2. Add a simple navigation bar using <nav>.
3. Include footer with author and copyright.

14. CSS Inclusion Methods & Basic Syntax

- Three ways to apply CSS: inline, internal, external.
- Use external CSS for maintainability and caching.

- CSS rule: selector { property: value; }
- Comments: /* comment */

Example

external.html

```
<link rel="stylesheet" href="style.css">
```

style.css

```
body { background: #f5f7fb; font-family: Arial, sans-serif; }
h1 { color: #003366; }
```

Practice tasks

1. Create three files (inline.html, internal.html, external.html) demonstrating each method; change the background color and font in each.
2. Explain benefits of external CSS (caching, reuse). Show browser DevTools Network tab to see style.css cached.
3. Add comments and split CSS into logical sections (Typography, Layout, Components).

15. Selectors, Combinators & Specificity

- **Basic selectors:** element (p), class (.note), id (#main), attribute (input[type="text"]).
- **Combinators:** descendant (div p), child (ul > li), adjacent sibling (h2 + p), general sibling (h2 ~ p). Pseudo-classes/elements: :hover, :focus, :nth-child(), ::before, ::after.
- **Specificity order:** inline style > ID > class/attribute/pseudo-class > element/pseudo-element. Later rules override earlier if same specificity.

Example

external.html

```
<p id="demo" class="test">Hello</p>
```

style.css

```
p { color: red; } /* element */
.test { color: green; } /* class */
```

Practice tasks

1. Create HTML with elements having both class and id; write conflicting rules and predict which color will render.
2. Use ::before and ::after to add decorative text to headings (content: "• ").
3. Use div + p and div ~ p to style only adjacent/general sibling paragraphs.

16. Text & Font Properties (CSS3)

- color, font-family, font-size, font-weight, font-style, line-height, letter-spacing, word-spacing, text-align, text-indent, text-transform, text-decoration.
- font-variant for small-caps. Use fallback font stacks. Use Google Fonts via <link>.

Example

style.css

```
h1 { font-family: 'Poppins', sans-serif; font-size: 36px; text-transform: uppercase; }
```

Good Practice tasks

1. Create a "Poem Page" demonstrating text-indent, line-height, letter-spacing.
2. Import a Google Font and apply to headings and body; show the fallback chain.
3. Create a "quote block" using <blockquote> and style with font-style: italic and ::before decorative quote mark.

17. Paragraphs, Indentation, Word-spacing & Alignment

- text-indent controls first-line indentation.
- Horizontal alignment: text-align: left|right|center|justify.
- Vertical spacing: line-height.
- word-spacing increases gaps between words.

Example

style.css

```
p { text-indent: 40px; text-align: justify; word-spacing: 2px; }
```

Practice tasks

1. Convert a plain paragraph into newspaper-style justified text with text-indent.
2. Show word-spacing difference between a default and increased value.
3. Create a short article with h2 headings and adjust spacing for readability.

18. Box Model, Units & Shorthand

- Box = content → padding → border → margin.
- box-sizing: content-box | border-box (recommend border-box for easier sizing).
- Units: absolute (px), relative (em, rem), percentage (%), viewport units (vw, vh, vmin, vmax). PDF introduces vmin/vmax — useful for responsive text/images.
- Shorthand properties: margin: 10px 5px;, border: 2px solid #000;, padding: 10px 20px;.

Example

style.css

```
* { box-sizing: border-box; }
.container { width: 80%; padding: 1rem; margin: 1rem auto; border: 1px solid #ccc; }
```

Practice tasks

1. Build two boxes side by side (floating or inline-block) and experiment with box-sizing.
2. Change widths using px, em, rem, %, and vw to observe behavior.
3. Draw and label a box-model diagram for a selected element.

19. Backgrounds, Gradients & Borders (Shorthand)

- background-color, background-image, background-repeat, background-position, background-size.
- CSS gradients: linear-gradient, radial-gradient.
- Borders: border-style, border-width, border-color, border-radius. Shorthand border: 2px solid #f00;. PDF lists many border styles (dashed, dotted, double, groove, ridge, inset, outset).
- border-radius supports individual corners and percentages.

Example

style.css

```
.header { background: linear-gradient(90deg, #0066ff, #00ccff); padding: 1rem; color: white; }
.card { border: 1px solid #ddd; border-radius: 8px; box-shadow: 0 2px 4px rgba(0,0,0,0.1); }
```

Practice tasks

1. Create three notice boxes with solid, dashed, double borders and different border-radius.
2. Use linear-gradient for page header background.
3. Create an image with a custom border color and rounded corners.

20. Lists & Table Styling

- list-style-type controls bullet style (disc, circle, square, none).
- Tables: border-collapse, border-spacing, th/td padding, text-align, caption-side. Use tr:nth-child(even) for zebra rows.

Example

style.css

```
ul { list-style-type: square; }
table { border-collapse: collapse; width: 100%; }
th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
tr:nth-child(even) { background: #f8f8f8; }
```

Practice tasks

1. Style a "Student Results" table with alternating colors and a styled caption.
2. Create a menu list with custom bullets and hover style for li:hover.
3. Use :first-child or :last-child to style specific list entries.

21. Images, <figure> & <figcaption>, float and wrapping text

- Image insertion: `<img src="" alt="" width="" height="">`. Prefer CSS sizing: width and height via CSS to allow responsive scaling.
- `<figure>` & `<figcaption>` provide semantic grouping for images.
- `float: left|right` causes text wrapping; use `clear/clearfix` pattern for parent containers.

Example

index.html

```
<figure class="thumb">
  
  <figcaption>Beautiful flower</figcaption>
</figure>
```

style.css

```
.thumb img { width: 200px; border-radius: 8px; box-shadow: 0 2px 6px rgba(0,0,0,.15);
float:left; margin: 0 15px 15px 0; }
.clearfix::after { content: ""; display: table; clear: both; }
```

Practice tasks

1. Build a 3-thumbnail gallery using `<figure>` and `figcaption`.
2. Float image left and show paragraph wrapping; then apply `.clearfix` to parent.
3. Use `max-width` and `height:auto` to ensure images scale gracefully.

22. Forms & Input Styling (Focus on UX)

- Base form controls: input, textarea, select, button. Target types with `input[type="text"]`, `input[type="submit"]`.
- Use `:focus` to highlight active fields. Use placeholder styling. PDF mentions `accent-color` (for checkboxes/radios/buttons) — modern addition.

Example

style.css

```
input, select, textarea { padding: .5rem; border: 1px solid #ccc; border-radius: 4px;
width: 100%; box-sizing: border-box; }
input:focus { border-color: #0078d4; outline: none; box-shadow: 0 0 0 3px
rgba(0,120,212,0.12); }
button { accent-color: #0078d4; }
```

Practice tasks

1. Style a registration form: aligned labels, inputs with padding and `:focus` effect.
2. Style checkboxes/radio using `accent-color` and test `:focus-visible`.
3. Create a custom button with hover and active states.

23. CSS3 Flexbox and Grid Layout

Flexbox (Flexible Box Layout)

- Designed for **one-dimensional layouts** — arranging items in a row or a column.
- Parent container uses `display: flex;`, and children adjust automatically.

Key properties:

Property	Description
<code>flex-direction</code>	Defines direction: row, column, row-reverse, column-reverse
<code>justify-content</code>	Horizontal alignment (flex-start, center, space-between, etc.)
<code>align-items</code>	Vertical alignment (flex-start, center, stretch, etc.)
<code>flex-wrap</code>	Wrap items to new lines
<code>gap</code>	Space between flex items

Example

index.html

```
<div class="container">
  <div class="box">Box 1</div>
  <div class="box">Box 2</div>
  <div class="box">Box 3</div>
</div>
```

style.css

```
.container {
  display: flex;
  justify-content: space-around;
  align-items: center;
  flex-wrap: wrap;
  background-color: #eef2f6;
  padding: 20px;
  gap: 20px;
}
.box {
  background-color: #0078d4;
  color: white;
  width: 150px;
  height: 100px;
  display: flex;
  justify-content: center;
  align-items: center;
  border-radius: 8px;
}
```

Grid Layout

- Designed for **two-dimensional layouts** — controlling rows and columns simultaneously.
- Parent container uses `display: grid;`.
- Key properties:

Property	Description
grid-template-columns	Defines column sizes (e.g., 1fr 1fr 1fr)
grid-template-rows	Defines row sizes
gap	Space between grid cells
grid-column / grid-row	Span items across rows/columns

Example

index.html

```
<div class="grid-container">
  <div class="grid-item">1</div>
  <div class="grid-item">2</div>
  <div class="grid-item">3</div>
  <div class="grid-item">4</div>
  <div class="grid-item">5</div>
  <div class="grid-item">6</div>
</div>
```

style.css

```
.grid-item {
  background-color: #ff7a59;
  color: white;
  text-align: center;
  padding: 20px 0;
  border-radius: 6px;
}
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 15px;
  background-color: #f7f7f7;
  padding: 20px;
}
```

24. Transforms, Transitions & Transitions properties

- transform (2D): translate(), rotate(), scale(), skew().
- transition shorthand properties: transition-property, transition-duration, transition-timing-function, transition-delay.
- Use will-change sparingly for performance hints.

Example

style.css

```
.button { transition: transform 0.25s ease, background-color 0.25s ease; }
.button:hover { transform: translateY(-4px) scale(1.02); background-color: #005fa3; }
```

Good Practice tasks

1. Create buttons that lift and change color on hover using transform + transition.
2. Animate an image scale on hover (thumbnail zoom effect).
3. Use transition-delay to sequence hover effects.

25. CSS3 Animations (@keyframes)

- @keyframes name { from { } to { } } or % key points.
- Animation properties: animation-name, animation-duration, animation-timing-function, animation-iteration-count, animation-direction, animation-delay, animation-fill-mode, animation-play-state.
- Advantages: offloads animation to browser compositor — often better than JS for simple animations.

Example

style.css

```
.box { animation: float 4s ease-in-out infinite; }
@keyframes float {
  0% { transform: translateY(0); }
  50% { transform: translateY(-20px); }
}
```

Practice tasks

1. Animate a banner logo with @keyframes that fades and moves.
2. Create a multi-step animation (color + position + scale).
3. Use animation-play-state toggled via a button (pause/play).

26. CSS3 Transitions vs Animations & Vendor Prefixes

- Transitions are for simple state changes (hover/focus). Animations are for keyframe sequences.
- Vendor prefixes historically required: -webkit-, -moz-, -o-. Modern browsers support many features unprefixed but check caniuse.com. Use autoprefixer in build workflows.

Practice tasks for home

1. Add vendor-prefixed variants for transform and animation and test in multiple browsers.
2. Use DevTools to check whether prefixes are needed (try older browsers if available).

27. Positioning (static, relative, absolute, fixed, sticky)

- position: static (default) — normal flow.
- relative — offsets relative to normal position.
- absolute — positioned relative to nearest positioned ancestor.
- fixed — viewport fixed (useful for sticky headers).
- sticky — toggles between relative and fixed based on scroll.

Example

style.css

```
.box { position: relative; }  
.box .label { position: absolute; top: 10px; right: 10px; }
```

Practice tasks

1. Build a card with an absolutely positioned badge (e.g., "NEW") inside a relative container.
2. Implement a simple header and demonstrate position: fixed behavior.
3. Use position: sticky to make a section header stick at top when scrolling.

28. Float and Clear (legacy but important)

- float moves element left/right and text wraps. Use for images and legacy two-column layouts.
- Clearing floats: clear:left|right|both or use .clearfix pattern. PDF example shows float: left for image wrap.

Practice tasks for home

1. Create a paragraph with a left-floated thumbnail and observe wrapping.
2. Implement .clearfix after a float container to avoid collapse.
3. Convert a float-based two-column layout into inline-block alternative (practice reasoning about layout choices).

29. Modern CSS3 Features

These are intermediate/advanced CSS3 features. Include demos, caveats about browser support, and how to progressively enhance.

- Viewport units: 1vw = 1% of viewport width, vh, vmin, vmax. Useful for fluid sizing.
- h1 { font-size: 4vw; }
- :has() — parent selector: article:has(img) { border-left: 4px solid #0078d4; } (check support).
- Cascade layers (@layer) — manage rule precedence: @layer base, components, utilities { ... }.
- accent-color — style form controls: input[type=checkbox] { accent-color: #0066ff; }.
- Scroll-timeline / scroll-driven animations — newer API to tie animations to scroll position (experimental).
- Nesting — now available with preprocessors and new spec (e.g., article { h2 { color:red } }).
- Subgrid — Grid Level 2 feature; allows alignment of children tracks with parent grid (support limited).
- Note: Use feature detection (@supports) and progressive enhancement.

Practice tasks for home

1. Use vw for a headline and compare with rem sizing.
2. Try input[type=checkbox]{accent-color: #28a745;} and test checkbox color.

3. Use @supports to provide fallback styles when :has() or subgrid is unavailable.

30. CSS3 Media Queries

Media Queries let CSS respond to the **characteristics of the device or viewport**, such as width, height, orientation, or media type.

Example

style.css

```
@media media-type and (condition) { /* CSS rules */ }
```

Common Media Types:

- screen → for monitors, tablets, and phones
- print → for printers; speech → for screen readers

Common Conditions:

- max-width — style applies when screen width is **less than or equal** to this value
- min-width — style applies when screen width is **greater than or equal** to this value
- orientation — portrait or landscape

Most common use: Create **responsive** text or layout changes for mobile, tablet, and desktop views.

Example

index.html

```
body {  
  font-family: Arial, sans-serif;  
  text-align: center;  
  padding: 50px;  
  background-color: lightgray;  
}  
h1 {  
  color: darkblue;  
}
```

style.css

```
/* Tablet screens */  
@media screen and (max-width: 768px) {  
  body {  
    background-color: lightblue;  
  }  
  h1 {  
    color: darkgreen;  
  }  
}  
/* Mobile screens */  
@media screen and (max-width: 480px) {  
  body {  
    background-color: lightpink;  
  }  
  h1 {
```

```
    color: crimson;
  }
}
/* Landscape mode */
@media screen and (orientation: landscape) {
  h1 {
    font-style: italic;
  }
}
```

Combine conditions

```
@media screen and (min-width:600px) and (max-width:900px) { ... }
```

Print media query example

```
@media print { body { color: black; background: white; } }
```

Lab Tasks

HTML: Create a personal portfolio website that includes:

- A header with your name/logo and a navigation bar linking to different sections/pages.
- A hero section with an introduction about yourself.
- A footer with contact info and social links.
- At least two additional pages about your favorite travel destinations, linked from the navbar.

CSS: Style your portfolio website pages using CSS:

- Apply a consistent color scheme and modern fonts across all pages.
- Style the navigation bar with hover effects, and ensure the active page is highlighted.
- Add styling to the hero section, footers, and travel pages, including images, cards, and text formatting.
- Use Flexbox or Grid for layout, transitions/hover effects for interactive elements, and make the site responsive for mobile, tablet, and desktop.

AI: Polish and improve your portfolio with AI:

- Initiate an AI agent in your local projects folder where the HTML and CSS file is situated
- Ask AI with good prompts to polish the portfolio you have developed, add more pages to it.

Git: Push your codes in GitHub using Git:

- Initialize a new Git repository in a folder, add the HTML and CSS file before AI polish, commit them, and push them in the main branch to the GitHub repository.
- After polish the codes with AI, add all the files again, commit them, and push them.